拓展优化开发文档

本文档记录了三项拓展优化开发内容，包含原来代码和拓展优化后的新代码对比，方便手动修改。

拓展优化一：开发 pb_max 最大化按钮功能（roomdialog.ui)

问题原因

roomdialog.ui 界面中的 pb_max 最大化按钮没有实际功能，点击无反应。同时 pb_min 最小化按钮也没有槽函数。

涉及文件

- 客户端：WeChat/roomdialog.h、WeChat/roomdialog.cpp

文件1：WeChat/roomdialog.h

原代码

```
private slots:
    void on_pb_close_clicked();

    void on_pb_quit_clicked();

    void closeEvent(QCloseEvent* event);

    void on_cb_audio_clicked();

    void on_cb_video_clicked();

    void on_cb_desk_clicked();

private:
    Ui::RoomDialog *ui;
    QVBoxLayout * m_mainLayout;
    std::map<int , UserShow*>m_mapIDToUserShow;
};
```

修改后代码

```
private slots:
    void on_pb_close_clicked();

    void on_pb_quit_clicked();

    void closeEvent(QCloseEvent* event);

    void on_cb_audio_clicked();

    void on_cb_video_clicked();

    void on_cb_desk_clicked();

    //【拓展优化-新增】最大化/还原按钮的槽函数
    //点击 pb_max 按钮时触发，在"窗口最大化"和"还原原始大小"两种状态间切换
    void on_pb_max_clicked();

    //【拓展优化-新增】最小化按钮的槽函数
    //点击 pb_min 按钮时触发，将窗口最小化到任务栏
    void on_pb_min_clicked();

private:
    Ui::RoomDialog *ui;
    QVBoxLayout * m_mainLayout;
    std::map<int , UserShow*>m_mapIDToUserShow;

    //【拓展优化-新增】保存窗口还原时的几何信息
    //用于在最大化与还原之间切换时恢复窗口原始位置和大小
    QRect m_normalGeometry;
    //【拓展优化-新增】标记当前是否处于最大化状态
    bool m_isMaximized;
};
```

文件2：WeChat/roomdialog.cpp

原代码（构造函数）

```
#include "roomdialog.h"
#include "ui_roomdialog.h"
#include <QMessageBox>
#include <QDebug>

RoomDialog::RoomDialog(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::RoomDialog)
{
```

```
    ui->setupUi(this);

    m_mainLayout = new QVBoxLayout;

    m_mainLayout->setContentsMargins(0,0,0,0);
    m_mainLayout->setSpacing(5);

    //设置一个竖直布局的画布,可以向这里添加控件
    ui->wdg_list->setLayout( m_mainLayout );

//    //测试
//    for(int i=0 ; i<6 ; ++i)
//    {
//        UserShow* user = new UserShow;
//        user->slot_setInfo(i+1 , QString("测试%1").arg(i+1));
//        slot_addUserShow( user );
//    }
}
```

修改后代码（构造函数）

```
#include "roomdialog.h"
#include "ui_roomdialog.h"
#include <QMessageBox>
#include <QDebug>
//【拓展优化-新增】最大化功能所需的头文件
#include <QApplication>
#include <QScreen>
#include <QWindow>

RoomDialog::RoomDialog(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::RoomDialog)
{
    ui->setupUi(this);

    m_mainLayout = new QVBoxLayout;

    m_mainLayout->setContentsMargins(0,0,0,0);
    m_mainLayout->setSpacing(5);

    //设置一个竖直布局的画布,可以向这里添加控件
    ui->wdg_list->setLayout( m_mainLayout );

    //【拓展优化-新增】初始化最大化状态标记为 false（未最大化）
    m_isMaximized = false;
    //【拓展优化-新增】保存窗口初始几何信息，用于还原
    m_normalGeometry = this->geometry();
}
```

原代码（on_pb_close_clicked 和 on_pb_quit_clicked）

```
//退出房间
void RoomDialog::on_pb_close_clicked()
{
    this->close();
}


void RoomDialog::on_pb_quit_clicked()
{
    this->close();
}
```

修改后代码（新增 on_pb_min_clicked 和 on_pb_max_clicked）

```
//退出房间
void RoomDialog::on_pb_close_clicked()
{
    this->close();
}


void RoomDialog::on_pb_quit_clicked()
{
    this->close();
}

//【拓展优化-新增】最小化按钮点击槽函数
//功能：将房间窗口最小化到任务栏
//实现：调用 showMinimized() 让窗口最小化
void RoomDialog::on_pb_min_clicked()
{
    this->showMinimized();
}

//【拓展优化-新增】最大化/还原按钮点击槽函数
//功能：在"窗口最大化"和"还原原始大小"两种状态间切换
//实现：
//  - 若当前未最大化(m_isMaximized == false)：
//      1. 保存当前窗口几何信息到 m_normalGeometry（用于后续还原）
//      2. 获取当前屏幕的可用几何区域（排除任务栏）
//      3. 将窗口设置为该屏幕的可用区域大小，实现最大化效果
//      4. 标记 m_isMaximized = true
//  - 若当前已最大化(m_isMaximized == true)：
//      1. 从 m_normalGeometry 恢复窗口原始位置和大小
//      2. 标记 m_isMaximized = false
void RoomDialog::on_pb_max_clicked()
{
```

```
    if( !m_isMaximized )
    {
        //保存当前窗口几何信息，用于还原
        m_normalGeometry = this->geometry();
        //获取窗口所在屏幕的可用几何区域（排除任务栏）
        QScreen * screen = QApplication::primaryScreen();
        if( screen )
        {
            QRect deskRect = screen->availableGeometry();
            this->setGeometry( deskRect );
        }
        m_isMaximized = true;
    }else{
        //还原窗口到原始位置和大小
        this->setGeometry( m_normalGeometry );
        m_isMaximized = false;
    }
}
```

修改要点

1. roomdialog.h 新增 on_pb_max_clicked() 和 on_pb_min_clicked() 槽函数声明
2. roomdialog.h 新增 m_normalGeometry 和 m_isMaximized 成员变量
3. roomdialog.cpp 构造函数初始化 m_isMaximized = false 和 m_normalGeometry
4. roomdialog.cpp 新增 on_pb_min_clicked() 实现（调用 showMinimized()）
5. roomdialog.cpp 新增 on_pb_max_clicked() 实现（切换最大化/还原状态）

拓展优化二：注册新增邮箱字段，登录支持手机号/用户名/邮箱三种方式

问题原因

1. 注册功能缺少邮箱字段
2. 登录只能通过手机号，无法通过用户名或邮箱登录

涉及文件

- 客户端：WeChat/netapi/net/packdef.h、WeChat/logindialog.h、WeChat/logindialog.cpp、WeChat/logindialog.ui、WeChat/ckernel.h、WeChat/ckernel.cpp
- 服务端：服务端代码/include/pack_def.h、服务端代码/src/clogic.cpp

文件1：客户端 WeChat/netapi/net/packdef.h

原代码（STRU_REGISTER_RQ）

```
//协议结构
//注册请求
typedef struct STRU_REGISTER_RQ
{
    STRU_REGISTER_RQ()
    {
        m_nType = DEF_PACK_REGISTER_RQ;
        memset( m_tel  , 0, MAX_SIZE);
        memset( m_name  , 0,MAX_SIZE);
        memset( m_password , 0,MAX_SIZE );
    }
    //需要手机号码 , 密码, 昵称
    PackType m_nType;//包类型
    char m_tel[MAX_SIZE];//电话号
    char m_name[MAX_SIZE];//昵称
    char m_password[MAX_SIZE];//密码

}STRU_REGISTER_RQ;
```

修改后代码（STRU_REGISTER_RQ）

```
//协议结构
//注册请求
typedef struct STRU_REGISTER_RQ
{
    STRU_REGISTER_RQ()
    {
        m_nType = DEF_PACK_REGISTER_RQ;
        memset( m_tel  , 0, MAX_SIZE);
        memset( m_name  , 0,MAX_SIZE);
        memset( m_password , 0,MAX_SIZE );
        //【拓展优化-新增】初始化邮箱字段
        memset( m_email , 0, MAX_SIZE );
    }
    //需要手机号码 , 密码, 昵称
    PackType m_nType;//包类型
    char m_tel[MAX_SIZE];//电话号
    char m_name[MAX_SIZE];//昵称
    char m_password[MAX_SIZE];//密码
    //【拓展优化-新增】邮箱字段
    //用于注册时存储用户邮箱，支持后续通过邮箱登录
    char m_email[MAX_SIZE];//邮箱

}STRU_REGISTER_RQ;
```

原代码（STRU_LOGIN_RQ）

```
//登录
typedef struct STRU_LOGIN_RQ
{
       //登录需要: 手机号 密码
    STRU_LOGIN_RQ()
       {
        m_nType = DEF_PACK_LOGIN_RQ;
        memset( m_tel, 0, MAX_SIZE );
        memset( m_password, 0, MAX_SIZE );
       }
    PackType m_nType;//包类型
    char m_tel[_MAX_SIZE];//用户ID
    char m_password[_MAX_SIZE];//密码

}STRU_LOGIN_RQ;
```

修改后代码（STRU_LOGIN_RQ）

```
//登录
typedef struct STRU_LOGIN_RQ
{
       //登录需要: 手机号 密码
    STRU_LOGIN_RQ()
       {
        m_nType = DEF_PACK_LOGIN_RQ;
        memset( m_tel, 0, MAX_SIZE );
        memset( m_password, 0, MAX_SIZE );
       }
    PackType m_nType;//包类型
    //【拓展优化-修改】登录标识字段
    //原字段 m_tel 仅支持手机号登录
    //现复用该字段作为通用登录标识，可传入手机号/用户名（昵称）/邮箱
    //服务端会自动识别类型并查询对应字段
    char m_tel[_MAX_SIZE];//登录标识（手机号/用户名/邮箱）
    char m_password[_MAX_SIZE];//密码

}STRU_LOGIN_RQ;
```

文件2：服务端 服务端代码/include/pack_def.h

原代码（STRU_REGISTER_RQ）

```
typedef struct STRU_REGISTER_RQ
{
```

```
    STRU_REGISTER_RQ()
    {
        m_nType = DEF_PACK_REGISTER_RQ;
        memset( m_tel  , 0, MAX_SIZE);
        memset( m_name  , 0,MAX_SIZE);
        memset( m_password , 0,MAX_SIZE );
    }
    //需要手机号码 , 密码, 昵称
    PackType m_nType;//包类型
    char m_tel[MAX_SIZE];//电话号
    char m_name[MAX_SIZE];//昵称
    char m_password[MAX_SIZE];//密码

}STRU_REGISTER_RQ;
```

修改后代码（STRU_REGISTER_RQ)

```
typedef struct STRU_REGISTER_RQ
{
    STRU_REGISTER_RQ()
    {
        m_nType = DEF_PACK_REGISTER_RQ;
        memset( m_tel  , 0, MAX_SIZE);
        memset( m_name  , 0,MAX_SIZE);
        memset( m_password , 0,MAX_SIZE );
        //【拓展优化-新增】初始化邮箱字段
        memset( m_email , 0, MAX_SIZE );
    }
    //需要手机号码 , 密码, 昵称
    PackType m_nType;//包类型
    char m_tel[MAX_SIZE];//电话号
    char m_name[MAX_SIZE];//昵称
    char m_password[MAX_SIZE];//密码
    //【拓展优化-新增】邮箱字段
    //用于注册时存储用户邮箱，支持后续通过邮箱登录
    char m_email[MAX_SIZE];//邮箱

}STRU_REGISTER_RQ;
```

原代码（STRU_LOGIN_RQ)

```
    STRU_LOGIN_RQ()
    {
        m_nType = DEF_PACK_LOGIN_RQ;
        memset( m_tel, 0, MAX_SIZE );
        memset( m_password, 0, MAX_SIZE );
    }
    PackType m_nType;//包类型
    char m_tel[_MAX_SIZE];//用户ID
```

```
    char m_password[_MAX_SIZE];//密码

}STRU_LOGIN_RQ;
```

修改后代码（STRU_LOGIN_RQ）

```
    STRU_LOGIN_RQ()
    {
        m_nType = DEF_PACK_LOGIN_RQ;
        memset( m_tel, 0, MAX_SIZE );
        memset( m_password, 0, MAX_SIZE );
    }
    PackType m_nType;//包类型
    //【拓展优化-修改】登录标识字段
    //原字段 m_tel 仅支持手机号登录
    //现复用该字段作为通用登录标识，可传入手机号/用户名（昵称）/邮箱
    //服务端会自动识别类型并查询对应字段
    char m_tel[_MAX_SIZE];//登录标识（手机号/用户名/邮箱）
    char m_password[_MAX_SIZE];//密码

}STRU_LOGIN_RQ;
```

文件3：服务端 服务端代码/src/clogic.cpp

原代码（RegisterRq 函数中的插入逻辑）

```
       else
       {
           //没有，寫表tel pass name頭像和籤名的默認值 返回注冊成功
           rs.m_lresult = register_success;
           sprintf(sqlStr , "insert into t_user( tel, password , name , icon , feeling) values('%s
                   ,rq->m_tel,rq->m_password,rq->m_name, 1 ,"比較懶，什麼也沒寫");
           if(!m_sql->UpdataMysql(sqlStr))
           {
               printf("UpdataMysql error: %s \n",sqlStr);
           }
       }
```

修改后代码（RegisterRq 函数中的插入逻辑）

```
       else
       {
           //【拓展优化-新增】检查邮箱是否已存在
           //原代码未检查邮箱唯一性，可能导致同一邮箱注册多个账号
           //修复：注册前检查邮箱是否已存在于数据库
           resList.clear();
```

```
        sprintf(sqlStr , "select email from t_user where email = '%s';",rq->m_email);
        if(!m_sql->SelectMysql(sqlStr , 1 , resList))
        {
            printf("SelectMysql error: %s \n",sqlStr);
            return;
        }
        if( resList.size() > 0)
        {
            //邮箱已存在，返回错误（复用 name_is_exist 错误码表示"已存在"）
            rs.m_lresult = name_is_exist;
        }
        else
        {
            //沒有，寫表tel pass name email 頭像和籤名的默認值 返回注冊成功
            rs.m_lresult = register_success;
            //【拓展优化-修改】INSERT 语句新增 email 字段
            //原代码只插入 tel, password, name, icon, feeling
            //修复：新增 email 字段插入
            sprintf(sqlStr , "insert into t_user( tel, password , name , email , icon , feeling
                    ,rq->m_tel,rq->m_password,rq->m_name, rq->m_email, 1 ,"比較懶，什麼也沒寫");
            if(!m_sql->UpdataMysql(sqlStr))
            {
                printf("UpdataMysql error: %s \n",sqlStr);
            }
        }
    }
```

原代码（LoginRq 函数中的查询逻辑）

```
char strsql[1024] = "";
list<string> lstRes;
sprintf( strsql , "select password,id,name from t_user where tel = '%s';",rq->m_tel);

if(!m_sql->SelectMysql(strsql , 3 , lstRes))
{
    printf("SelectMysql error: %s \n",strsql);
    return;
}
```

修改后代码（LoginRq 函数中的查询逻辑）

```
char strsql[1024] = "";
list<string> lstRes;
//【拓展优化-修改】支持手机号/用户名/邮箱三种方式登录
//原代码只能通过手机号登录：where tel = '...'
//修复：根据输入特征自动识别登录类型
//  - 包含 '@' → 邮箱登录，查询 email 字段
//  - 全数字且长度11位 → 手机号登录，查询 tel 字段
//  - 其他 → 用户名（昵称）登录，查询 name 字段
```

```
    string strLoginId = rq->m_tel;
    string strField = "tel"; //默认按手机号查询
    if( strLoginId.find('@') != string::npos )
    {
        //包含@符号，识别为邮箱
        strField = "email";
    }
    else if( strLoginId.find_first_not_of("0123456789") == string::npos && strLoginId.length() == 1
    {
        //全数字且11位，识别为手机号
        strField = "tel";
    }
    else
    {
        //其他情况，识别为用户名（昵称）
        strField = "name";
    }
    sprintf( strsql , "select password,id,name from t_user where %s = '%s';",strField.c_str(), rq->

    if(!m_sql->SelectMysql(strsql , 3 , lstRes))
    {
        printf("SelectMysql error: %s \n",strsql);
        return;
    }
```

文件4：客户端 WeChat/logindialog.h

原代码

```
signals:
    void SIG_loginCommit(QString tel , QString password);
    void SIG_registerCommit(QString tel , QString password , QString name );
    void SIG_close();
```

修改后代码

```
signals:
    void SIG_loginCommit(QString tel , QString password);
    //【拓展优化-修改】注册信号新增 email 参数
    //原信号：SIG_registerCommit(QString tel, QString password, QString name)
    //修改后：SIG_registerCommit(QString tel, QString password, QString name, QString email)
    void SIG_registerCommit(QString tel , QString password , QString name , QString email );
    void SIG_close();
```

文件5：客户端 WeChat/logindialog.cpp

原代码（on_pb_commit_clicked 登录校验）

```
#include <QRegExp>
#include <QMessageBox>
//提交登录信息
void LoginDialog::on_pb_commit_clicked()
{
    QString strTel      = ui->le_tel->text();
    QString strPassword = ui->le_password->text();
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty())
    {
        QMessageBox::about(this, "提示", "手机号密码不能为空格");
        return;
    }
    //1.校验手机号 正则表达式
    QRegExp reg("^1[3-9][0-9]{9}$");
    bool res = reg.exactMatch(strTel);
    if(!res)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }

    //2.校验密码 长度不能超过20
    if( strPassword.length() > 20)
    {
        QMessageBox::about(this, "提示", "密码过长，长度不超过20");
        return;
    }
    Q_EMIT SIG_loginCommit( strTel , strPassword );
    qDebug()<<__func__;
}
```

修改后代码（on_pb_commit_clicked 登录校验）

```
#include <QRegExp>
#include <QMessageBox>
//提交登录信息
void LoginDialog::on_pb_commit_clicked()
{
    QString strTel      = ui->le_tel->text();
    QString strPassword = ui->le_password->text();
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty())
```

```
{
    QMessageBox::about(this, "提示", "手机号密码不能为空格");
    return;
}
//【拓展优化-修改】登录校验逻辑支持手机号/用户名/邮箱三种方式
//原代码仅校验手机号格式：^1[3-9][0-9]{9}$
//修改后：
//  - 包含 '@' → 邮箱格式校验
//  - 全数字且11位 → 手机号格式校验
//  - 其他 → 用户名校验（非空且长度<=10）
bool bValid = false;
if( strTel.contains('@') )
{
    //邮箱登录：简单校验邮箱格式
    QRegExp regEmail("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    bValid = regEmail.exactMatch(strTel);
    if(!bValid)
    {
        QMessageBox::about(this, "提示", "邮箱格式错误");
        return;
    }
}
else if( strTel.length() == 11 && !strTel.contains(QRegExp("[^0-9]")) )
{
    //手机号登录：校验手机号格式
    QRegExp reg("^1[3-9][0-9]{9}$");
    bValid = reg.exactMatch(strTel);
    if(!bValid)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }
}
else
{
    //用户名（昵称）登录：校验长度
    if( strTel.length() > 10 || strTel.length() < 1 )
    {
        QMessageBox::about(this, "提示", "用户名长度需在1-10之间");
        return;
    }
    bValid = true;
}

//2.校验密码 长度不能超过20
if( strPassword.length() > 20)
{
    QMessageBox::about(this, "提示", "密码过长，长度不超过20");
    return;
}
Q_EMIT SIG_loginCommit( strTel , strPassword );
```

```
    qDebug()<<__func__;
}
```

原代码（on_pb_clear_register_clicked 清空注册信息）

```
//清空注册信息
void LoginDialog::on_pb_clear_register_clicked()
{
    ui->le_confirm_register->setText("");
    ui->le_name_register->setText("");
    ui->le_password_register->setText("");
    ui->le_tel_register->setText("");

}
```

修改后代码（on_pb_clear_register_clicked 清空注册信息）

```
//清空注册信息
void LoginDialog::on_pb_clear_register_clicked()
{
    ui->le_confirm_register->setText("");
    ui->le_name_register->setText("");
    ui->le_password_register->setText("");
    ui->le_tel_register->setText("");
    //【拓展优化-新增】清空邮箱输入框
    ui->le_email_register->setText("");

}
```

原代码（on_pb_commit_register_clicked 注册提交）

```
//注册提交
void LoginDialog::on_pb_commit_register_clicked()
{

    QString strTel      = ui->le_tel_register->text();
    QString strPassword = ui->le_password_register->text();
    QString strConfirm  = ui->le_confirm_register->text();
    QString strName     = ui->le_name_register->text();
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword, tmpName = strName;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty() || tmpName.remove(" ").isEmpt
    {
        QMessageBox::about(this, "提示", "手机号密码昵称不能为空格");
        return;
    }
```

```
    //1.校验手机号 正则表达式
    QRegExp reg("^1[3-8][0-9]\{6,9\}$");
    bool res = reg.exactMatch(strTel);
    if(!res)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }

    //2.校验密码 长度不能超过20
    if( strPassword.length() > 20)
    {
        QMessageBox::about(this, "提示", "密码过长，长度不超过20");
        return;
    }
    //3.两次输入是否一致
    if( strPassword != strConfirm)
    {
        QMessageBox::about(this, "提示", "两次输入的密码不一致");
        return;
    }
    //4.name长度不超过10
    if(strName.length() > 10)
    {
        QMessageBox::about(this, "提示", "昵称长度不能超过10");
        return;
    }
    Q_EMIT SIG_registerCommit( strTel , strPassword , strName );
    qDebug()<<__func__;

}
```

修改后代码（on_pb_commit_register_clicked 注册提交）

```
//注册提交
void LoginDialog::on_pb_commit_register_clicked()
{

    QString strTel      = ui->le_tel_register->text();
    QString strPassword = ui->le_password_register->text();
    QString strConfirm  = ui->le_confirm_register->text();
    QString strName     = ui->le_name_register->text();
    //【拓展优化-新增】获取邮箱输入
    QString strEmail    = ui->le_email_register->text();
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword, tmpName = strName, tmpEmail = strEmail;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty() || tmpName.remove(" ").isEmpt
    {
        QMessageBox::about(this, "提示", "手机号密码昵称邮箱不能为空格");
```

```
        return;
    }
    //1.校验手机号 正则表达式
    QRegExp reg("^1[3-8][0-9]\{6,9\}$");
    bool res = reg.exactMatch(strTel);
    if(!res)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }

    //2.校验密码 长度不能超过20
    if( strPassword.length() > 20)
    {
        QMessageBox::about(this, "提示", "密码过长，长度不超过20");
        return;
    }
    //3.两次输入是否一致
    if( strPassword != strConfirm)
    {
        QMessageBox::about(this, "提示", "两次输入的密码不一致");
        return;
    }
    //4.name长度不超过10
    if(strName.length() > 10)
    {
        QMessageBox::about(this, "提示", "昵称长度不能超过10");
        return;
    }
    //【拓展优化-新增】校验邮箱格式
    QRegExp regEmail("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    if(!regEmail.exactMatch(strEmail))
    {
        QMessageBox::about(this, "提示", "邮箱格式错误");
        return;
    }
    //【拓展优化-修改】注册信号新增 email 参数
    Q_EMIT SIG_registerCommit( strTel , strPassword , strName , strEmail );
    qDebug()<<__func__;

}
```

文件6：客户端 WeChat/logindialog.ui

原代码（le_name_register 后面）

```
    <widget class="QLineEdit" name="le_name_register">
     <property name="geometry">
      <rect>
```

```
      <x>130</x>
      <y>30</y>
      <width>150</width>
      <height>30</height>
     </rect>
    </property>
    <property name="toolTip">
     <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p&gt;输入10位以内昵称&lt;/p&gt;&lt;/body&gt;
    </property>
   </widget>
  </widget>
```

修改后代码（新增邮箱标签和输入框）

```
   <widget class="QLineEdit" name="le_name_register">
    <property name="geometry">
     <rect>
      <x>130</x>
      <y>30</y>
      <width>150</width>
      <height>30</height>
     </rect>
    </property>
    <property name="toolTip">
     <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p&gt;输入10位以内昵称&lt;/p&gt;&lt;/body&gt;
    </property>
   </widget>
   <!--【拓展优化-新增】邮箱标签和输入框，用于注册时填写邮箱 -->
   <widget class="QLabel" name="lb_email_register">
    <property name="geometry">
     <rect>
      <x>20</x>
      <y>230</y>
      <width>71</width>
      <height>30</height>
     </rect>
    </property>
    <property name="font">
     <font>
      <pointsize>10</pointsize>
     </font>
    </property>
    <property name="text">
     <string>邮箱</string>
    </property>
   </widget>
   <widget class="QLineEdit" name="le_email_register">
    <property name="geometry">
     <rect>
      <x>130</x>
```

```
        <y>230</y>
        <width>150</width>
        <height>30</height>
       </rect>
      </property>
      <property name="toolTip">
       <string>&lt;html&gt;&lt;head/&gt;&lt;body&gt;&lt;p&gt;输入有效邮箱，可用于登录&lt;/p&gt;&lt;/b
      </property>
     </widget>
    </widget>
```

文件7：客户端 WeChat/ckernel.h

原代码

```
    //发送注册信息
    void slot_registerCommit(QString tel, QString pass, QString name);
```

修改后代码

```
    //发送注册信息
    //【拓展优化-修改】新增 email 参数，用于注册时传递邮箱
    void slot_registerCommit(QString tel, QString pass, QString name, QString email);
```

文件8：客户端 WeChat/ckernel.cpp

原代码（connect 绑定）

```
    connect(m_pLoginDlg , SIGNAL(SIG_registerCommit(QString , QString, QString)) , this , SLOT(slo
```

修改后代码（connect 绑定）

```
    //【拓展优化-修改】注册信号绑定新增 email 参数
    connect(m_pLoginDlg , SIGNAL(SIG_registerCommit(QString , QString, QString, QString)) , this ,
```

原代码（slot_registerCommit 函数）

```
//发送注册信息
void ckernel::slot_registerCommit(QString tel, QString pass, QString name)
{
    qDebug() << "[DEBUG] slot_registerCommit 被调用, tel:" << tel << " name:" << name;
```

```
        //修复：检查网络连接状态，未连接时提示用户，避免 SendData 内部阻塞重连导致 UI 卡死
        if( !m_pClient || !m_pClient->IsConnected() )
        {
            qDebug() << "[DEBUG] 网络未连接";
            QMessageBox::about(m_pLoginDlg, "提示", "网络未连接，请检查服务器是否启动后重试");
            return;
        }
        qDebug() << "[DEBUG] 网络已连接，准备发送注册请求";
    std::string strTel = tel.toStdString();
    //中文
    std::string strName = name.toStdString();//格式utf-8

    STRU_REGISTER_RQ rq;
    //【安全修复】使用 strncpy 替代 strcpy，防止缓冲区溢出
    //STRU_REGISTER_RQ 中 m_tel/m_name/m_password 均为 char[MAX_SIZE](60字节)
    strncpy(rq.m_tel , strTel.c_str() , sizeof(rq.m_tel) - 1);
    rq.m_tel[sizeof(rq.m_tel) - 1] = '\0';

    std::string strPassMD5 = GetMD5(pass);
    qDebug()<<strPassMD5.c_str();
    //兼容中文，使用 strncpy 防止昵称字段溢出
    strncpy(rq.m_name , strName.c_str() , sizeof(rq.m_name) - 1);
    rq.m_name[sizeof(rq.m_name) - 1] = '\0';

    strncpy(rq.m_password , strPassMD5.c_str() , sizeof(rq.m_password) - 1);
    rq.m_password[sizeof(rq.m_password) - 1] = '\0';

    /*先转换std，后转换char**/
    /*发送*/
    m_pClient->SendData(0 , (char*)&rq, sizeof(rq));

}
```

修改后代码（slot_registerCommit 函数）

```
//发送注册信息
//【拓展优化-修改】函数签名新增 email 参数
//原函数：slot_registerCommit(QString tel, QString pass, QString name)
//修改后：slot_registerCommit(QString tel, QString pass, QString name, QString email)
void ckernel::slot_registerCommit(QString tel, QString pass, QString name, QString email)
{
    qDebug() << "[DEBUG] slot_registerCommit 被调用, tel:" << tel << " name:" << name << " email:"
        //修复：检查网络连接状态，未连接时提示用户，避免 SendData 内部阻塞重连导致 UI 卡死
        if( !m_pClient || !m_pClient->IsConnected() )
        {
            qDebug() << "[DEBUG] 网络未连接";
            QMessageBox::about(m_pLoginDlg, "提示", "网络未连接，请检查服务器是否启动后重试");
            return;
        }
        qDebug() << "[DEBUG] 网络已连接，准备发送注册请求";
```

```
    std::string strTel = tel.toStdString();
    //中文
    std::string strName = name.toStdString();//格式utf-8
    //【拓展优化-新增】获取邮箱字符串
    std::string strEmail = email.toStdString();

    STRU_REGISTER_RQ rq;
    //【安全修复】使用 strncpy 替代 strcpy，防止缓冲区溢出
    //STRU_REGISTER_RQ 中 m_tel/m_name/m_password 均为 char[MAX_SIZE](60字节)
    strncpy(rq.m_tel , strTel.c_str() , sizeof(rq.m_tel) - 1);
    rq.m_tel[sizeof(rq.m_tel) - 1] = '\0';

    std::string strPassMD5 = GetMD5(pass);
    qDebug()<<strPassMD5.c_str();
    //兼容中文，使用 strncpy 防止昵称字段溢出
    strncpy(rq.m_name , strName.c_str() , sizeof(rq.m_name) - 1);
    rq.m_name[sizeof(rq.m_name) - 1] = '\0';

    strncpy(rq.m_password , strPassMD5.c_str() , sizeof(rq.m_password) - 1);
    rq.m_password[sizeof(rq.m_password) - 1] = '\0';

    //【拓展优化-新增】填充邮箱字段
    strncpy(rq.m_email , strEmail.c_str() , sizeof(rq.m_email) - 1);
    rq.m_email[sizeof(rq.m_email) - 1] = '\0';

    /*先转换std，后转换char**/
    /*发送*/
    m_pClient->SendData(0 , (char*)&rq, sizeof(rq));

}
```

修改要点

1. packdef.h（客户端和服务端）STRU_REGISTER_RQ 新增 m_email 字段
2. packdef.h（客户端和服务端）STRU_LOGIN_RQ 的 m_tel 字段注释改为"登录标识（手机号/用户名/邮箱）"
3. 服务端 clogic.cpp 的 RegisterRq 新增邮箱唯一性检查和 INSERT 语句新增 email 字段
4. 服务端 clogic.cpp 的 LoginRq 根据输入特征自动识别登录类型（手机号/用户名/邮箱）
5. 客户端 logindialog.h 的 SIG_registerCommit 信号新增 email 参数
6. 客户端 logindialog.cpp 的 on_pb_commit_clicked 支持三种登录方式校验
7. 客户端 logindialog.cpp 的 on_pb_commit_register_clicked 新增邮箱校验和传递
8. 客户端 logindialog.ui 新增邮箱标签和输入框
9. 客户端 ckernel.h 的 slot_registerCommit 新增 email 参数
10. 客户端 ckernel.cpp 的 connect 绑定和 slot_registerCommit 实现新增 email 参数

数据库修改提示

需要给 t_user 表新增 email 字段：

```
ALTER TABLE t_user ADD COLUMN email VARCHAR(60) DEFAULT '';
```

拓展优化三：开发 pb_set 设置按钮和 toolButton_3 预定会议功能

问题原因

wechatdialog.ui 界面中的 pb_set 设置按钮和 toolButton_3 预定会议按钮没有实际功能，点击无反应。

涉及文件

- 客户端：WeChat/wechatdialog.h、WeChat/wechatdialog.cpp

文件1：WeChat/wechatdialog.h

原代码

```
class WeChatDialog : public QDialog
{
    Q_OBJECT
signals:
    void SIG_close();
    void SIG_createRoom();
    void SIG_joinRoom();


public:
    WeChatDialog(QWidget *parent = nullptr);
    ~WeChatDialog();

    void closeEvent(QCloseEvent *event);

    void setInfo(QString  name, int icon = 1);

private slots:
    void on_pb_create_clicked();

    void on_pb_join_clicked();
```

```
private:
    Ui::WeChatDialog *ui;
};
```

修改后代码

```
class WeChatDialog : public QDialog
{
    Q_OBJECT
signals:
    void SIG_close();
    void SIG_createRoom();
    void SIG_joinRoom();
    //【拓展优化-新增】预定会议信号
    //点击"预定会议"按钮时触发，由 ckernel 接收处理
    void SIG_bookRoom();
    //【拓展优化-新增】设置信号
    //点击"设置"按钮时触发，由 ckernel 接收处理
    void SIG_settings();


public:
    WeChatDialog(QWidget *parent = nullptr);
    ~WeChatDialog();

    void closeEvent(QCloseEvent *event);

    void setInfo(QString  name, int icon = 1);

private slots:
    void on_pb_create_clicked();

    void on_pb_join_clicked();

    //【拓展优化-新增】预定会议按钮点击槽函数
    void on_toolButton_3_clicked();

    //【拓展优化-新增】设置按钮点击槽函数
    void on_pb_set_clicked();

private:
    Ui::WeChatDialog *ui;
};
```

文件2：WeChat/wechatdialog.cpp

原代码（头文件包含）

```
#include "wechatdialog.h"
#include "ui_wechatdialog.h"
#include <QMessageBox>
#include <QDebug>
```

修改后代码（头文件包含）

```
#include "wechatdialog.h"
#include "ui_wechatdialog.h"
#include <QMessageBox>
#include <QDebug>
//【拓展优化-新增】预定会议和设置功能所需的头文件
#include <QDateTime>
#include <QInputDialog>
#include <QSettings>
#include <QCoreApplication>
#include <QDir>
```

原代码（on_pb_join_clicked 后面没有其他槽函数）

```
void WeChatDialog::on_pb_join_clicked()
{
    Q_EMIT SIG_joinRoom();

}
```

修改后代码（新增 on_toolButton_3_clicked 和 on_pb_set_clicked）

```
void WeChatDialog::on_pb_join_clicked()
{
    Q_EMIT SIG_joinRoom();

}

//【拓展优化-新增】预定会议按钮点击槽函数
//功能：预定一个未来的会议，生成会议号和会议时间，保存到本地配置文件
//实现：
//  1. 弹出输入对话框让用户输入会议主题
//  2. 弹出输入对话框让用户选择会议时长（分钟）
//  3. 生成唯一的会议号（基于当前时间戳）
//  4. 将会议信息（会议号、主题、时间、时长）保存到本地配置文件 booking.ini
//  5. 显示预定成功提示，包含会议号和会议时间
void WeChatDialog::on_toolButton_3_clicked()
{
    //1.输入会议主题
    bool ok = false;
```

```
QString topic = QInputDialog::getText(this, tr("预定会议"),
                                      tr("请输入会议主题:"), QLineEdit::Normal,
                                      tr("我的会议"), &ok);
if( !ok || topic.isEmpty() )
{
    return;
}

//2.输入会议时长（分钟）
QStringList items;
items << tr("30分钟") << tr("60分钟") << tr("90分钟") << tr("120分钟");
QString item = QInputDialog::getItem(this, tr("预定会议"),
                                      tr("请选择会议时长:"), items, 0, false, &ok);
if( !ok || item.isEmpty() )
{
    return;
}

//解析时长
int duration = 30;
if( item.startsWith("60") ) duration = 60;
else if( item.startsWith("90") ) duration = 90;
else if( item.startsWith("120") ) duration = 120;

//3.生成会议号（基于当前时间戳，确保唯一性）
//格式：MMDDHHMM（月日时分）
qint64 timestamp = QDateTime::currentDateTime().toSecsSinceEpoch();
int roomid = (int)(timestamp % 100000000);  //取后8位

//4.计算会议开始时间（当前时间 + 5分钟提醒缓冲）
QDateTime startTime = QDateTime::currentDateTime().addSecs(5 * 60);
QString strStartTime = startTime.toString("yyyy-MM-dd HH:mm");

//5.保存到本地配置文件 booking.ini
//配置文件保存在程序所在目录下，方便用户查看
QString configPath = QCoreApplication::applicationDirPath() + "/booking.ini";
QSettings settings(configPath, QSettings::IniFormat);
settings.beginGroup("Booking_" + QString::number(roomid));
settings.setValue("roomid", roomid);
settings.setValue("topic", topic);
settings.setValue("starttime", strStartTime);
settings.setValue("duration", duration);
settings.endGroup();

//6.显示预定成功提示
QString info = tr("会议预定成功！\n\n")
             + tr("会议号：%1\n").arg(roomid)
             + tr("会议主题：%1\n").arg(topic)
             + tr("开始时间：%1\n").arg(strStartTime)
             + tr("会议时长：%1分钟\n").arg(duration)
             + tr("\n请记下会议号，到时间后使用\"加入会议\"功能输入会议号加入。");
QMessageBox::information(this, tr("预定成功"), info);
```

```
    //7.发送信号通知 ckernel（预留扩展，可用于服务端同步）
    Q_EMIT SIG_bookRoom();
}

//【拓展优化-新增】设置按钮点击槽函数
//功能：打开设置对话框，让用户配置音频/视频设备、网络等参数
//实现：
//  1. 从本地配置文件 settings.ini 读取当前设置
//  2. 弹出多页输入对话框让用户修改设置
//  3. 保存设置到本地配置文件
//  4. 发送信号通知 ckernel 应用新设置
void WeChatDialog::on_pb_set_clicked()
{
    //1.读取当前设置
    QString configPath = QCoreApplication::applicationDirPath() + "/settings.ini";
    QSettings settings(configPath, QSettings::IniFormat);

    //当前设置项
    QString curServerIp = settings.value("Network/ServerIP", "127.0.0.1").toString();
    int curVideoBitrate = settings.value("Video/Bitrate", 500).toInt();
    int curAudioVolume = settings.value("Audio/Volume", 80).toInt();

    //2.弹出设置对话框（使用输入对话框简化实现）
    bool ok = false;

    //2.1 服务器IP设置
    QString serverIp = QInputDialog::getText(this, tr("设置 - 服务器"),
                                             tr("服务器IP地址:"), QLineEdit::Normal,
                                             curServerIp, &ok);
    if( !ok ) return;

    //2.2 视频码率设置
    QStringList videoItems;
    videoItems << tr("低 (300kbps)") << tr("中 (500kbps)") << tr("高 (800kbps)");
    int curVideoIndex = (curVideoBitrate <= 300) ? 0 : ((curVideoBitrate <= 500) ? 1 : 2);
    QString videoItem = QInputDialog::getItem(this, tr("设置 - 视频"),
                                              tr("视频码率:"), videoItems, curVideoIndex, false, &ok);
    if( !ok ) return;

    int videoBitrate = 500;
    if( videoItem.startsWith("低") ) videoBitrate = 300;
    else if( videoItem.startsWith("中") ) videoBitrate = 500;
    else if( videoItem.startsWith("高") ) videoBitrate = 800;

    //2.3 音频音量设置
    int audioVolume = QInputDialog::getInt(this, tr("设置 - 音频"),
                                           tr("音频音量 (0-100):"), curAudioVolume,
                                           0, 100, 1, &ok);
    if( !ok ) return;

    //3.保存设置到配置文件
```

```
    settings.setValue("Network/ServerIP", serverIp);
    settings.setValue("Video/Bitrate", videoBitrate);
    settings.setValue("Audio/Volume", audioVolume);

    //4.提示用户
    QMessageBox::information(this, tr("设置"),
        tr("设置已保存，部分设置重启程序后生效。\n\n")
        + tr("服务器IP: %1\n").arg(serverIp)
        + tr("视频码率: %1 kbps\n").arg(videoBitrate)
        + tr("音频音量: %1\n").arg(audioVolume));

    //5.发送信号通知 ckernel
    Q_EMIT SIG_settings();
}
```

修改要点

1. wechatdialog.h 新增 SIG_bookRoom() 和 SIG_settings() 信号
2. wechatdialog.h 新增 on_toolButton_3_clicked() 和 on_pb_set_clicked() 槽函数
3. wechatdialog.cpp 新增头文件包含（QDateTime、QInputDialog、QSettings、QCoreApplication、QDir）
4. wechatdialog.cpp 新增 on_toolButton_3_clicked() 实现（预定会议功能）
5. wechatdialog.cpp 新增 on_pb_set_clicked() 实现（设置功能）

功能说明

预定会议功能

- 用户点击"预定会议"按钮后，依次弹出输入对话框
- 输入会议主题和选择会议时长（30/60/90/120分钟）
- 自动生成唯一的会议号（基于时间戳）
- 将会议信息保存到程序目录下的 booking.ini 文件
- 显示预定成功提示，包含会议号和会议时间
- 用户可使用"加入会议"功能输入会议号加入预定的会议

设置功能

- 用户点击"设置"按钮后，依次弹出输入对话框
- 可设置服务器IP地址、视频码率（低/中/高）、音频音量（0-100）
- 将设置保存到程序目录下的 settings.ini 文件
- 显示设置保存成功提示
- 部分设置（如服务器IP）需要重启程序后生效

手动修改步骤

客户端修改步骤

1. **roomdialog.h**：在 `private slots:` 末尾新增 `on_pb_max_clicked()` 和 `on_pb_min_clicked()` 声明；在 `private:` 末尾新增 `QRect m_normalGeometry;` 和 `bool m_isMaximized;`
2. **roomdialog.cpp**：新增头文件包含；构造函数初始化 `m_isMaximized` 和 `m_normalGeometry`；新增 `on_pb_min_clicked()` 和 `on_pb_max_clicked()` 实现
3. **netapi/net/packdef.h**：`STRU_REGISTER_RQ` 新增 `m_email` 字段；`STRU_LOGIN_RQ` 的 `m_tel` 注释改为"登录标识"
4. **logindialog.h**：`SIG_registerCommit` 信号新增 `QString email` 参数
5. **logindialog.cpp**：`on_pb_commit_clicked` 支持三种登录方式校验；`on_pb_clear_register_clicked` 清空邮箱；`on_pb_commit_register_clicked` 新增邮箱校验和传递
6. **logindialog.ui**：在 `le_name_register` 后新增 `lb_email_register` 和 `le_email_register`
7. **ckernel.h**：`slot_registerCommit` 新增 `QString email` 参数
8. **ckernel.cpp**：connect 绑定新增 email 参数；`slot_registerCommit` 实现新增 email 参数
9. **wechatdialog.h**：新增 `SIG_bookRoom()` 和 `SIG_settings()` 信号；新增 `on_toolButton_3_clicked()` 和 `on_pb_set_clicked()` 槽函数
10. **wechatdialog.cpp**：新增头文件包含；新增 `on_toolButton_3_clicked()` 和 `on_pb_set_clicked()` 实现

服务端修改步骤

1. **include/pack_def.h**：`STRU_REGISTER_RQ` 新增 `m_email` 字段；`STRU_LOGIN_RQ` 的 `m_tel` 注释改为"登录标识"
2. **src/clogic.cpp**：
   - `RegisterRq` 函数：新增邮箱唯一性检查；INSERT 语句新增 email 字段
   - `LoginRq` 函数：根据输入特征自动识别登录类型（手机号/用户名/邮箱）

数据库修改步骤

执行以下 SQL 语句给 t_user 表新增 email 字段：

```
ALTER TABLE t_user ADD COLUMN email VARCHAR(60) DEFAULT '';
```

拓展优化四：已注册用户补全邮箱功能

问题原因

已注册的用户在数据库中没有邮箱信息。当这类用户登录时，需要判断邮箱是否为空，若为空则弹出对话框让用户补全邮箱信息。

涉及文件

- 服务端：服务端代码/include/packdef.h、服务端代码/include/clogic.h、服务端代码/src/clogic.cpp
- 客户端：WeChat/netapi/net/packdef.h、WeChat/ckernel.h、WeChat/ckernel.cpp

---

文件1：服务端 服务端代码/include/packdef.h

原代码

```
//登录请求的结果
#define user_not_exist              (0)
#define password_error              (1)
#define login_success        (2)
```

修改后代码

```
//登录请求的结果
#define user_not_exist              (0)
#define password_error              (1)
#define login_success        (2)
//【拓展优化-新增】登录成功但邮箱为空，需要补全邮箱
#define login_success_no_email (3)

//【拓展优化-新增】补全邮箱请求和回复
#define DEF_PACK_UPDATE_EMAIL_RQ  (DEF_PACK_BASE + 15)
#define DEF_PACK_UPDATE_EMAIL_RS  (DEF_PACK_BASE + 16)

//【拓展优化-新增】补全邮箱请求
//已注册用户登录后若邮箱为空，客户端弹出对话框让用户补全邮箱
//用户提交后通过此包发送给服务端
struct STRU_UPDATE_EMAIL_RQ
{
    STRU_UPDATE_EMAIL_RQ()
```

```
    {
        m_nType = DEF_PACK_UPDATE_EMAIL_RQ;
        m_userid = 0;
        memset( m_email , 0, MAX_SIZE );
    }
    PackType m_nType;//包类型
    int m_userid;//用户ID
    char m_email[MAX_SIZE];//邮箱
};

//【拓展优化-新增】补全邮箱回复
struct STRU_UPDATE_EMAIL_RS
{
    STRU_UPDATE_EMAIL_RS()
    {
        m_nType = DEF_PACK_UPDATE_EMAIL_RS;
        m_lresult = 0;
    }
    PackType m_nType;//包类型
    int m_lresult;//补全结果 1=成功 0=失败
};
```

文件2：客户端 WeChat/netapi/net/packdef.h

修改后代码

与服务端 packdef.h 同步，新增 login_success_no_email、DEF_PACK_UPDATE_EMAIL_RQ/RS 以及 STRU_UPDATE_EMAIL_RQ/RS 结构体（内容与服务端完全一致）。

文件3：服务端 服务端代码/include/clogic.h

原代码

```
    //音频注册
    void AudioRegister(int clientfd, char * szbuf, int nlen);
    //视频注册
    void VideoRegister(int clientfd, char * szbuf, int nlen);
```

修改后代码

```
    //音频注册
    void AudioRegister(int clientfd, char * szbuf, int nlen);
    //视频注册
```

```
    void VideoRegister(int clientfd, char * szbuf, int nlen);
    //【拓展优化-新增】补全邮箱
    //已注册用户登录后若邮箱为空，客户端弹出对话框让用户补全邮箱
    //用户提交后通过此函数处理
    void UpdateEmailRq(int clientfd, char * szbuf, int nlen);
```

文件4：服务端 服务端代码/src/clogic.cpp

修改1：setNetPackMap 新增协议映射

原代码

```
    NetPackMap(DEF_PACK_AUDIO_REGISTER)      =&CLogic::AudioRegister;
    NetPackMap(DEF_PACK_VIDEO_REGISTER)      =&CLogic::VideoRegister;

}
```

修改后代码

```
    NetPackMap(DEF_PACK_AUDIO_REGISTER)      =&CLogic::AudioRegister;
    NetPackMap(DEF_PACK_VIDEO_REGISTER)      =&CLogic::VideoRegister;
    //【拓展优化-新增】注册补全邮箱协议映射
    NetPackMap(DEF_PACK_UPDATE_EMAIL_RQ)     =&CLogic::UpdateEmailRq;

}
```

修改2：LoginRq 函数 SQL 查询新增 email 字段

原代码

```
sprintf( strsql , "select password,id,name from t_user where %s = '%s';",strField.c_str(), rq->m_t
//...
if(!m_sql->SelectMysql(strsql , 3 , lstRes))
```

修改后代码

```
//【拓展优化-修改】SQL 查询新增 email 字段
//原代码：select password,id,name from t_user where ...
//修改后：select password,id,name,email from t_user where ...
//用于登录成功后判断邮箱是否为空，若为空返回 login_success_no_email
sprintf( strsql , "select password,id,name,email from t_user where %s = '%s';",strField.c_str(), r
//...
if(!m_sql->SelectMysql(strsql , 4 , lstRes))
```

修改3：LoginRq 函数 登录成功后判断邮箱

原代码

```
//验证密码
if( strcmp(rq->m_password, strPwd.c_str()) != 0 )
{
    rs.m_lresult = password_error;
}else{
    //密码正确
    int id = atoi(strId.c_str());
    //sock要保存起來 id->sock映射（爲了通信）
    UserInfo * pInfo = new UserInfo;
    pInfo->m_id = id;
    pInfo->m_roomid = 0;
    pInfo->m_sock = clientfd;
    strcpy(pInfo->m_userName , strName.c_str());

    //判斷id是否在線，在線強制下線，不在線添加
    if(m_mapIDToUserInfo.IsExist(pInfo->m_id))
    {
        //強制下線（原代码未实现，暂保留）
        m_mapIDToUserInfo.erase(pInfo->m_id);
    }
    m_mapIDToUserInfo.insert(pInfo->m_id , pInfo);
    //寫返回包 返回 帶id和結果
    rs.m_userid = id;
    strcpy(rs.m_name , pInfo->m_userName);
    rs.m_lresult = login_success;
}
```

修改后代码

```
//验证密码
if( strcmp(rq->m_password, strPwd.c_str()) != 0 )
{
    rs.m_lresult = password_error;
}else{
    //密码正确
    int id = atoi(strId.c_str());
    //sock要保存起來 id->sock映射（爲了通信）
    UserInfo * pInfo = new UserInfo;
    pInfo->m_id = id;
    pInfo->m_roomid = 0;
    pInfo->m_sock = clientfd;
    strcpy(pInfo->m_userName , strName.c_str());

    //判斷id是否在線，在線強制下線，不在線添加
    if(m_mapIDToUserInfo.IsExist(pInfo->m_id))
```

```
            {
                //強制下線（原代码未实现，暂保留）
                m_mapIDToUserInfo.erase(pInfo->m_id);
            }
            m_mapIDToUserInfo.insert(pInfo->m_id , pInfo);
            //寫返回包 返回 帶id和結果
            rs.m_userid = id;
            strcpy(rs.m_name , pInfo->m_userName);
            //【拓展优化-新增】判断邮箱是否为空
            //若邮箱为空，返回 login_success_no_email，客户端收到后弹出补全邮箱对话框
            //若邮箱不为空，返回 login_success
            if( strEmail.empty() )
            {
                rs.m_lresult = login_success_no_email;
            }
            else
            {
                rs.m_lresult = login_success;
            }
        }
```

修改4：新增 UpdateEmailRq 函数

新增代码

```
//【拓展优化-新增】补全邮箱请求处理
//功能：已注册用户登录后若邮箱为空，客户端弹出对话框让用户补全邮箱
//用户提交后通过此函数更新数据库中的 email 字段
//流程：
//  1. 拆包获取 userid 和 email
//  2. 校验邮箱格式（简单校验：包含 @ 和 .）
//  3. 检查邮箱是否已被其他用户使用
//  4. 更新 t_user 表中该 userid 的 email 字段
//  5. 返回补全结果
void CLogic::UpdateEmailRq(int clientfd, char * szbuf, int nlen)
{
    printf("clientfd:%d UpdateEmailRq\n", clientfd);
    //1.拆包
    STRU_UPDATE_EMAIL_RQ * rq = (STRU_UPDATE_EMAIL_RQ *)szbuf;
    STRU_UPDATE_EMAIL_RS rs;

    //2.简单校验邮箱格式（包含 @ 和 .）
    string strEmail = rq->m_email;
    if( strEmail.find('@') == string::npos || strEmail.find('.') == string::npos )
    {
        rs.m_lresult = 0; //失败：邮箱格式错误
        SendData(clientfd, (char*)&rs, sizeof(rs));
        return;
    }
```

```
    //3.检查邮箱是否已被其他用户使用
    char sqlStr[1024] = {0};
    list<string> resList;
    sprintf(sqlStr, "select id from t_user where email = '%s' and id != %d;", rq->m_email, rq->m_u
    if(!m_sql->SelectMysql(sqlStr, 1, resList))
    {
        printf("SelectMysql error: %s \n", sqlStr);
        rs.m_lresult = 0;
        SendData(clientfd, (char*)&rs, sizeof(rs));
        return;
    }
    if(resList.size() > 0)
    {
        //邮箱已被其他用户使用
        rs.m_lresult = 0;
        SendData(clientfd, (char*)&rs, sizeof(rs));
        return;
    }

    //4.更新 t_user 表中该 userid 的 email 字段
    sprintf(sqlStr, "update t_user set email = '%s' where id = %d;", rq->m_email, rq->m_userid);
    if(!m_sql->UpdataMysql(sqlStr))
    {
        printf("UpdataMysql error: %s \n", sqlStr);
        rs.m_lresult = 0;
        SendData(clientfd, (char*)&rs, sizeof(rs));
        return;
    }

    //5.返回补全成功
    rs.m_lresult = 1;
    SendData(clientfd, (char*)&rs, sizeof(rs));
}
```

文件5：客户端 WeChat/ckernel.h

原代码

```
    //视频帧处理
    void slot_dealVideoFrameRq(uint sock,char* buf,int nlen);
```

修改后代码

```
    //视频帧处理
    void slot_dealVideoFrameRq(uint sock,char* buf,int nlen);
    //【拓展优化-新增】补全邮箱回复处理
    void slot_dealUpdateEmailRS(uint sock,char* buf,int nlen);
```

文件6：客户端 WeChat/ckernel.cpp

修改1：setNetPackMap 新增协议映射

原代码

```
    NetPackMap(DEF_PACK_AUDIO_FRAME)      = &ckernel::slot_dealAudioFrameRq;
    NetPackMap(DEF_PACK_VIDEO_FRAME)      = &ckernel::slot_dealVideoFrameRq;
}
```

修改后代码

```
    NetPackMap(DEF_PACK_AUDIO_FRAME)      = &ckernel::slot_dealAudioFrameRq;
    NetPackMap(DEF_PACK_VIDEO_FRAME)      = &ckernel::slot_dealVideoFrameRq;
    //【拓展优化-新增】补全邮箱回复映射
    NetPackMap(DEF_PACK_UPDATE_EMAIL_RS)  = &ckernel::slot_dealUpdateEmailRS;
}
```

修改2：slot_dealLoginRS 新增 login_success_no_email 分支

原代码

```
    case login_success:
    {
        //...登录成功处理...
    }
        break;
    default:
        break;
    }
}
```

修改后代码

```
    case login_success:
    {
        //...登录成功处理（不变）...
    }
        break;
    //【拓展优化-新增】登录成功但邮箱为空，需要补全邮箱
    case login_success_no_email:
    {
        QString strName = QString::fromUtf8( rs->m_name );
        QString strMsg = QString("用户[%1]登录成功\n\n您的账号尚未填写邮箱，请补全邮箱信息。").arg( str
```

```
        QMessageBox::about(m_pLoginDlg , "提示" , strMsg);

        m_id = rs->m_userid;
        m_UserName = QString::fromUtf8( rs->m_name );

        //弹出补全邮箱对话框，循环直到用户输入有效邮箱或取消
        bool ok = false;
        while( true )
        {
            QString email = QInputDialog::getText(m_pLoginDlg, tr("补全邮箱"),
                tr("请输入您的邮箱地址（可用于后续登录）:"),
                QLineEdit::Normal, "", &ok);
            if( !ok || email.isEmpty() )
            {
                //用户取消，直接进入主界面（不强制补全）
                break;
            }
            //简单校验邮箱格式
            QRegExp regEmail("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
            if( !regEmail.exactMatch(email) )
            {
                QMessageBox::about(m_pLoginDlg, "提示", "邮箱格式错误，请重新输入");
                continue;
            }
            //发送补全邮箱请求到服务端
            STRU_UPDATE_EMAIL_RQ rq;
            rq.m_userid = m_id;
            std::string strEmail = email.toStdString();
            strncpy(rq.m_email, strEmail.c_str(), sizeof(rq.m_email) - 1);
            rq.m_email[sizeof(rq.m_email) - 1] = '\0';
            m_pClient->SendData(0, (char*)&rq, sizeof(rq));
            //提示用户
            QMessageBox::about(m_pLoginDlg, "提示", "邮箱已提交，服务端验证中...");
            break;
        }

        //ui跳转
        m_pLoginDlg->hide();
        m_pWeChatDlg->showNormal();

        //注册音频和视频的fd
        STRU_AUDIO_REGISTER rq_audio;
        rq_audio.m_userid = m_id;
        STRU_VIDEO_REGISTER rq_video;
        rq_video.m_userid = m_id;
        m_pAVClient[audio_client]->SendData( 0 , (char*)&rq_audio , sizeof (rq_audio));
        m_pAVClient[video_client]->SendData( 0 , (char*)&rq_video , sizeof (rq_video));
    }
        break;
    default:
        break;
```

```
    }
}
```

修改3：新增 slot_dealUpdateEmailRS 函数

新增代码

```
//【拓展优化-新增】补全邮箱回复处理
//功能：处理服务端返回的补全邮箱结果
//成功：提示用户补全成功
//失败：提示用户补全失败（邮箱格式错误或已被使用）
void ckernel::slot_dealUpdateEmailRS(uint sock, char *buf, int nlen)
{
    qDebug()<<__func__;
    //拆包
    STRU_UPDATE_EMAIL_RS * rs = (STRU_UPDATE_EMAIL_RS *)buf;
    if( rs->m_lresult == 1 )
    {
        QMessageBox::about(m_pWeChatDlg, "提示", "邮箱补全成功！");
    }
    else
    {
        QMessageBox::about(m_pWeChatDlg, "提示", "邮箱补全失败，邮箱格式错误或已被使用");
    }
}
```

修改要点

1. **服务端 packdef.h**：新增 `login_success_no_email` 登录结果值；新增 `DEF_PACK_UPDATE_EMAIL_RQ/RS` 协议类型和对应结构体
2. **客户端 packdef.h**：与服务端同步新增相同定义
3. **服务端 clogic.h**：新增 `UpdateEmailRq` 函数声明
4. **服务端 clogic.cpp**：
   - `setNetPackMap` 注册 `DEF_PACK_UPDATE_EMAIL_RQ` 映射
   - `LoginRq` 函数 SQL 查询新增 email 字段，登录成功后判断邮箱是否为空
   - 新增 `UpdateEmailRq` 函数实现（校验格式、检查唯一性、更新数据库）
5. **客户端 ckernel.h**：新增 `slot_dealUpdateEmailRS` 槽函数声明
6. **客户端 ckernel.cpp**：
   - `setNetPackMap` 注册 `DEF_PACK_UPDATE_EMAIL_RS` 映射
   - `slot_dealLoginRS` 新增 `login_success_no_email` 分支，弹出补全邮箱对话框
   - 新增 `slot_dealUpdateEmailRS` 函数处理补全结果

拓展优化五：登录方式改为下拉框选择

问题原因

原登录功能通过自动识别输入特征判断登录方式（包含@为邮箱、全数字为手机号、其他为用户名），用户无法明确选择登录方式。改为下拉框让用户主动选择，体验更直观。

涉及文件

- 客户端：`WeChat/logindialog.ui`、`WeChat/logindialog.h`、`WeChat/logindialog.cpp`

文件1：WeChat/logindialog.ui

修改内容

在 `lb_tel` 标签上方新增"登录方式"标签和下拉框 `cb_login_type`，包含三个选项：手机号、用户名（昵称）、邮箱。

新增代码

```
<!--【拓展优化-新增】登录方式标签和下拉框，供用户选择登录方式 -->
<widget class="QLabel" name="lb_login_type">
 <property name="geometry">
  <rect>
   <x>20</x>
   <y>5</y>
   <width>71</width>
   <height>30</height>
  </rect>
 </property>
 <property name="font">
  <font>
   <pointsize>10</pointsize>
  </font>
 </property>
 <property name="text">
  <string>登录方式</string>
 </property>
</widget>
<widget class="QComboBox" name="cb_login_type">
 <property name="geometry">
  <rect>
   <x>110</x>
   <y>5</y>
```

```
    <width>150</width>
    <height>30</height>
   </rect>
  </property>
  <item>
   <property name="text">
    <string>手机号</string>
   </property>
  </item>
  <item>
   <property name="text">
    <string>用户名（昵称）</string>
   </property>
  </item>
  <item>
   <property name="text">
    <string>邮箱</string>
   </property>
  </item>
 </widget>
```

文件2：WeChat/logindialog.h

原代码

```
private slots:
    void on_pb_commit_clicked();
    void on_pb_clear_register_clicked();
    void on_pb_commit_register_clicked();
    void on_pb_clear_clicked();
```

修改后代码

```
private slots:
    void on_pb_commit_clicked();
    void on_pb_clear_register_clicked();
    void on_pb_commit_register_clicked();
    void on_pb_clear_clicked();
    //【拓展优化-新增】登录方式下拉框切换槽函数
    //当下拉框选择不同登录方式时，动态修改 lb_tel 标签的提示文字
    void on_cb_login_type_currentIndexChanged(int index);
```

文件3：WeChat/logindialog.cpp

修改1：on_pb_commit_clicked 根据下拉框选择进行校验

原代码

```
//【拓展优化-修改】登录校验逻辑支持手机号/用户名/邮箱三种方式
//原代码仅校验手机号格式：^1[3-9][0-9]{9}$
//修改后：
//  - 包含 '@' → 邮箱格式校验
//  - 全数字且11位 → 手机号格式校验
//  - 其他 → 用户名校验（非空且长度<=10）
bool bValid = false;
if( strTel.contains('@') )
{
    //邮箱登录：简单校验邮箱格式
    QRegExp regEmail("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    bValid = regEmail.exactMatch(strTel);
    if(!bValid)
    {
        QMessageBox::about(this, "提示", "邮箱格式错误");
        return;
    }
}
else if( strTel.length() == 11 && !strTel.contains(QRegExp("[^0-9]")) )
{
    //手机号登录：校验手机号格式
    QRegExp reg("^1[3-9][0-9]{9}$");
    bValid = reg.exactMatch(strTel);
    if(!bValid)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }
}
else
{
    //用户名（昵称）登录：校验长度
    if( strTel.length() > 10 || strTel.length() < 1 )
    {
        QMessageBox::about(this, "提示", "用户名长度需在1-10之间");
        return;
    }
    bValid = true;
}
```

修改后代码

```
//【拓展优化-修改】根据下拉框选择的登录方式进行不同的校验
//原代码仅校验手机号格式：^1[3-9][0-9]{9}$
//修改后：用户通过 cb_login_type 下拉框选择登录方式
//  - 索引0：手机号登录，校验 ^1[3-9][0-9]{9}$
//  - 索引1：用户名（昵称）登录，校验长度1-10
//  - 索引2：邮箱登录，校验邮箱格式
```

```
int loginType = ui->cb_login_type->currentIndex();
if( loginType == 0 )
{
    //手机号登录：校验手机号格式
    QRegExp reg("^1[3-9][0-9]{9}$");
    if(!reg.exactMatch(strTel))
    {
        QMessageBox::about(this, "提示", "手机号格式错误，应为11位手机号");
        return;
    }
}
else if( loginType == 1 )
{
    //用户名（昵称）登录：校验长度
    if( strTel.length() > 10 || strTel.length() < 1 )
    {
        QMessageBox::about(this, "提示", "用户名长度需在1-10之间");
        return;
    }
}
else if( loginType == 2 )
{
    //邮箱登录：校验邮箱格式
    QRegExp regEmail("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    if(!regEmail.exactMatch(strTel))
    {
        QMessageBox::about(this, "提示", "邮箱格式错误");
        return;
    }
}
```

修改2：新增 on_cb_login_type_currentIndexChanged 槽函数

新增代码

```
//【拓展优化-新增】登录方式下拉框切换槽函数
//功能：当下拉框选择不同登录方式时，动态修改 lb_tel 标签的提示文字
//  - 索引0：手机号 → lb_tel 显示"手机号"
//  - 索引1：用户名 → lb_tel 显示"用户名"
//  - 索引2：邮箱 → lb_tel 显示"邮箱"
void LoginDialog::on_cb_login_type_currentIndexChanged(int index)
{
    switch(index)
    {
    case 0:
        ui->lb_tel->setText("手机号");
        break;
    case 1:
        ui->lb_tel->setText("用户名");
        break;
```

```
    case 2:
        ui->lb_tel->setText("邮箱");
        break;
    default:
        break;
    }
}
```

修改要点

1. **logindialog.ui**：在登录页面顶部新增"登录方式"标签 `lb_login_type` 和下拉框 `cb_login_type`，包含三个选项
2. **logindialog.h**：新增 `on_cb_login_type_currentIndexChanged` 槽函数声明
3. **logindialog.cpp**：
   - `on_pb_commit_clicked` 改为根据 `cb_login_type->currentIndex()` 选择校验方式
   - 新增 `on_cb_login_type_currentIndexChanged` 槽函数，切换时动态修改 `lb_tel` 提示文字

拓展优化六：修复最大化后布局问题

问题原因

`roomdialog.ui` 中 `wdg_bottom` 控件（包含音频/视频/桌面共享复选框和退出房间按钮）使用绝对定位（geometry y=610）放置在 `wdg_UserShow` 内部。当窗口最大化时，`wdg_UserShow` 尺寸变大，但 `wdg_bottom` 仍然固定在 y=610 位置，不会自动跟随到底部，导致布局错乱。

涉及文件

- 客户端：`WeChat/roomdialog.ui`

文件1：WeChat/roomdialog.ui

原代码

```
<item>
 <widget class="QWidget" name="wdg_left" native="true">
  <layout class="QHBoxLayout" name="horizontalLayout_4">
   <item>
```

```
    <widget class="UserShow" name="wdg_UserShow" native="true">
     <property name="minimumSize">
      <size>
       <width>762</width>
       <height>540</height>
      </size>
     </property>
     <widget class="QWidget" name="wdg_bottom" native="true">
      <property name="geometry">
       <rect>
        <x>0</x>
        <y>610</y>
        <width>800</width>
        <height>40</height>
       </rect>
      </property>
      <!-- ... wdg_bottom 内部的复选框、按钮等控件 ... -->
     </widget>
    </widget>
   </item>
  </layout>
 </widget>
</item>
```

修改后代码

```
<item>
 <widget class="QWidget" name="wdg_left" native="true">
  <!--【拓展优化-修改】原布局为 QHBoxLayout(horizontalLayout_4)，wdg_bottom 使用绝对定位放在 wdg_UserS
      导致窗口最大化时 wdg_bottom 不会自动跟随到底部
      修改后：改为 QVBoxLayout(verticalLayout_left)，wdg_UserShow 设置 sizePolicy=Expanding 自动拉伸
      wdg_bottom 放在 wdg_UserShow 之后，自动贴底-->
  <layout class="QVBoxLayout" name="verticalLayout_left">
   <property name="spacing">
    <number>0</number>
   </property>
   <property name="leftMargin">
    <number>0</number>
   </property>
   <property name="topMargin">
    <number>0</number>
   </property>
   <property name="rightMargin">
    <number>0</number>
   </property>
   <property name="bottomMargin">
    <number>0</number>
   </property>
   <item>
    <widget class="UserShow" name="wdg_UserShow" native="true">
```

```
     <property name="sizePolicy">
      <sizepolicy hsizetype="Expanding" vsizetype="Expanding">
       <horstretch>0</horstretch>
       <verstretch>0</verstretch>
      </sizepolicy>
     </property>
     <property name="minimumSize">
      <size>
       <width>762</width>
       <height>540</height>
      </size>
     </property>
    </widget>
   </item>
   <item>
    <widget class="QWidget" name="wdg_bottom" native="true">
     <property name="minimumSize">
      <size>
       <width>0</width>
       <height>40</height>
      </size>
     </property>
     <property name="maximumSize">
      <size>
       <width>16777215</width>
       <height>40</height>
      </size>
     </property>
     <!-- ... wdg_bottom 内部的复选框、按钮等控件保持不变 ... -->
    </widget>
   </item>
  </layout>
 </widget>
</item>
```

修改要点

1. **wdg_left 布局类型**：从 QHBoxLayout（horizontalLayout_4）改为 QVBoxLayout（verticalLayout_left），使子控件垂直排列
2. **wdg_UserShow 设置 sizePolicy**：设置为 Expanding/Expanding，当窗口最大化时自动等比例拉伸填充空间
3. **wdg_bottom 移出 wdg_UserShow**：从 wdg_UserShow 的子控件（绝对定位）改为 wdg_left 布局的第二个 item，通过垂直布局自动放置在 wdg_UserShow 下方
4. **wdg_bottom 去掉 geometry**：不再使用绝对定位的 geometry 属性，改为通过布局管理器自动定位
5. **wdg_bottom minimumSize width 改为 0**：原为 800（固定宽度），改为 0 允许自适应宽度

修复效果

- 窗口正常大小时：wdg_UserShow 在上，wdg_bottom 在下，布局与原来一致
- 窗口最大化时：wdg_UserShow 自动等比例拉伸填充空间，wdg_bottom 自动跟随到底部，所有控件保持正确布局

手动修改步骤（新增）

客户端修改步骤（新增）

11. **netapi/net/packdef.h**：新增 login_success_no_email、DEF_PACK_UPDATE_EMAIL_RQ/RS、STRU_UPDATE_EMAIL_RQ/RS
12. **ckernel.h**：新增 slot_dealUpdateEmailRS 槽函数声明
13. **ckernel.cpp**：
    - setNetPackMap 新增 DEF_PACK_UPDATE_EMAIL_RS 映射
    - slot_dealLoginRS 新增 login_success_no_email 分支
    - 新增 slot_dealUpdateEmailRS 函数
14. **logindialog.ui**：新增 lb_login_type 标签和 cb_login_type 下拉框
15. **logindialog.h**：新增 on_cb_login_type_currentIndexChanged 槽函数声明
16. **logindialog.cpp**：
    - on_pb_commit_clicked 改为根据下拉框选择校验方式
    - 新增 on_cb_login_type_currentIndexChanged 槽函数
17. **roomdialog.ui**：wdg_left 布局从 QHBoxLayout 改为 QVBoxLayout；wdg_bottom 从 wdg_UserShow 内部移出到 wdg_left 布局；wdg_UserShow 设置 sizePolicy=Expanding

服务端修改步骤（新增）

3. **include/packdef.h**：新增 login_success_no_email、DEF_PACK_UPDATE_EMAIL_RQ/RS、STRU_UPDATE_EMAIL_RQ/RS
4. **include/clogic.h**：新增 UpdateEmailRq 函数声明
5. **src/clogic.cpp**：
    - setNetPackMap 新增 DEF_PACK_UPDATE_EMAIL_RQ 映射
    - LoginRq 函数 SQL 查询新增 email 字段，登录成功后判断邮箱是否为空
    - 新增 UpdateEmailRq 函数实现